

GitHub Actions Practical — CI/CD with Spring Boot

GitHub Actions allows us to automate tasks directly from a GitHub repository.

Instead of maintaining a separate CI/CD orchestration server such as Jenkins, GitHub can react to repository events and automatically execute workflows.

```
GitHub Repository
    +
Automation Engine
    =
GitHub Actions
```

1. What is GitHub Actions?

GitHub Actions is GitHub's automation platform that executes workflows in response to events.

It is commonly used for CI/CD, but it is not limited to CI/CD.

Examples:

```
Pull Request Created
    ↓
Run Tests
```

```
Release Created
    ↓
Build Docker Image
```

```
Issue Opened
    ↓
Automatically Add Label
```

```
Scheduled Time
    ↓
Run Maintenance Task
```

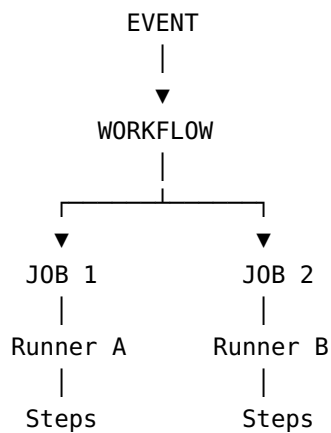
So CI/CD is simply one important use case of GitHub Actions.

2. The GitHub Actions Mental Model

The basic hierarchy is:

```
EVENT
  ↓
WORKFLOW
  ↓
JOB
  ↓
STEPS
  ↓
COMMAND / ACTION
```

A **runner** is responsible for actually executing the job.



Almost every GitHub Actions feature—caching, matrix builds, secrets, reusable workflows, OIDC, etc.—builds on top of this model.

3. Workflow

A **workflow** represents the complete automation process that GitHub should execute.

For example:

Whenever code is pushed to `main`, build and test the Spring Boot application.

That entire process can be one workflow.

A repository can contain multiple workflows:

```
.github/  
└─ workflows/  
    ├── ci.yml  
    ├── deploy.yml  
    └─ security-scan.yml
```

GitHub looks inside:

```
.github/workflows/
```

for workflow YAML files.

4. A Minimal GitHub Actions Workflow

```
name: My First Workflow  
  
on:  
  push:  
  
jobs:  
  hello:  
    runs-on: ubuntu-latest  
  
    steps:  
      - name: Say Hello  
        run: echo "Hello from GitHub Actions"
```

Think about the configuration in plain English:

```
name:  
What should this automation be called?  
  
on:  
When should it start?
```

```
jobs:
What work needs to be performed?

runs-on:
Which machine should perform the work?

steps:
What should that machine actually do?
```

5. `name`

```
name: My First Workflow
```

`name` is simply the human-readable name of the workflow.

It appears under the **Actions** tab of the GitHub repository.

It does not affect the execution logic.

6. Events and `on`

Automation needs a trigger.

GitHub calls the occurrence that starts a workflow an **event**.

```
Event
↓
Workflow
```

For example:

```
on:
  push:
```

means:

Run this workflow whenever a push event occurs.

GitHub supports many events, but some of the most common are:

Push

```
on:  
  push:
```

```
git push  
↓  
Workflow Runs
```

Pull Request

```
on:  
  pull_request:
```

A very common CI scenario:

```
Developer Opens PR  
↓  
Tests Automatically Run  
↓  
Merge Only if Tests Pass
```

Manual Trigger

```
on:  
  workflow_dispatch:
```

This allows a workflow to be started manually from the GitHub UI.

It is especially useful for testing and deployments.

Scheduled Trigger

```
on:
  schedule:
    - cron: '0 2 * * *'
```

This allows workflows to run at scheduled times.

7. Triggering Workflows for Specific Branches

Instead of reacting to every push:

```
on:
  push:
```

we can restrict the workflow:

```
on:
  push:
    branches:
      - main
```

Now:

```
Push → feature/login
      ↓
No Workflow

Push → main
      ↓
Workflow Runs
```

This becomes useful when CI and deployment have different rules.

For example:

```
Feature Branch / PR
      ↓
Run Tests
```

```
main
  ↓
Build + Deploy
```

8. Jobs

A workflow contains one or more **jobs**.

A job represents a unit of work inside the workflow.

Example:

Workflow: CI Pipeline

```
├─ Build Job
├─ Test Job
└─ Security Job
```

In YAML:

```
jobs:

  build:
    ...

  test:
    ...

  security:
    ...
```

9. Runner

A job requires a real machine to execute commands.

If the workflow contains:

```
mvn test
```

then Maven needs:

- CPU
- Memory
- Disk
- Operating system
- Java

That machine is called a **runner**.

A runner is the machine that executes a GitHub Actions job.

Example:

```
runs-on: ubuntu-latest
```

means GitHub should provide an Ubuntu runner for the job.

```
Job
↓
Needs CPU / RAM / OS
↓
Runner
```

10. GitHub Actions vs Jenkins

The concepts map closely to Jenkins.

Jenkins	GitHub Actions
Jenkinsfile	Workflow YAML
Pipeline	Workflow
Stage / Pipeline organization	Job / Steps
Agent / Node	Runner
<code>sh</code>	<code>run</code>
Credentials	Secrets

At a high level, both systems solve a similar problem:

```
"When something happens,  
give me a machine,  
and execute these commands."
```

11. GitHub-Hosted Runners

Example:

```
runs-on: ubuntu-latest
```

GitHub provides and maintains the machine.

Common options include:

```
runs-on: ubuntu-latest
```

```
runs-on: windows-latest
```

```
runs-on: macos-latest
```

Advantages of GitHub-hosted runners:

- No server management
- No manual setup
- Fresh environments
- Easy scaling
- GitHub maintains the infrastructure

They are an excellent choice for normal CI workloads.

12. Self-Hosted Runners

Sometimes a company needs jobs to execute on its own machines.

Reasons may include:

- Access to an internal database
- Private Maven repository
- Private network
- Special hardware
- GPU workloads
- Large custom machines
- Security requirements

In that case, we can register our own machine as a **self-hosted runner**.

GitHub



Your Machine

GitHub Runner Agent
Java
Maven
Internal Network
Company Tools

Basic syntax:

```
runs-on: self-hosted
```

Labels can also be used:

```
runs-on: [self-hosted, linux, x64]
```

GitHub will look for a connected runner matching those labels.

13. Steps

A job contains multiple **steps**.

Example:

```
Job: Build Application
```

Step 1 → Checkout Code
Step 2 → Setup Java
Step 3 → Run Maven

Example YAML:

```
jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v7

      - name: Build
        run: mvn package
```

All steps inside the same job execute on the same runner.

ONE RUNNER

```
graph TD
    A[Step 1: Checkout] --> B[Step 2: Setup Java]
    B --> C[Step 3: mvn package]
```

This means files created by an earlier step are available to later steps in the same job.

14. `run` vs `uses`

This distinction is fundamental in GitHub Actions.

`run`

`run` executes a shell command.

```
- name: Build
  run: mvn clean package
```

This is equivalent to opening a terminal and running:

```
mvn clean package
```

uses

`uses` executes a reusable GitHub Action.

Example:

```
- name: Checkout Repository
  uses: actions/checkout@v7
```

Instead of manually implementing repository checkout logic, we reuse an existing action.

15. What is an Action?

An **Action** is a reusable piece of automation that can be executed as a workflow step.

Examples:

```
actions/checkout
actions/setup-java
```

Do not confuse:

```
GitHub Actions
```

with:

```
An Action
```

The complete hierarchy is:

GitHub Actions



Workflow



Job



Step



Action

16. Why Reusable Actions are Useful

Without an action, setting up Java might require manually:

Download JDK



Extract JDK



Configure JAVA_HOME



Update PATH



Verify Installation

Instead:

```
- name: Setup Java
  uses: actions/setup-java@v6
  with:
    distribution: temurin
    java-version: '21'
```

The reusable action already knows how to configure Java correctly.

17. with

`with` provides inputs to an action.

Think of an action like a function.

In Java:

```
setupJava("temurin", 21);
```

In GitHub Actions:

```
uses: actions/setup-java@v6
with:
  distribution: temurin
  java-version: '21'
```

Conceptually:

```
uses
=
Which reusable action?

with
=
What inputs should be passed to it?
```

18. Spring Boot CI Workflow

A simple Spring Boot workflow can look like this:

```
name: Spring Boot CI

on:
  push:
    branches:
      - main

  pull_request:
    branches:
      - main

jobs:
  build:
```

```
runs-on: ubuntu-latest

steps:

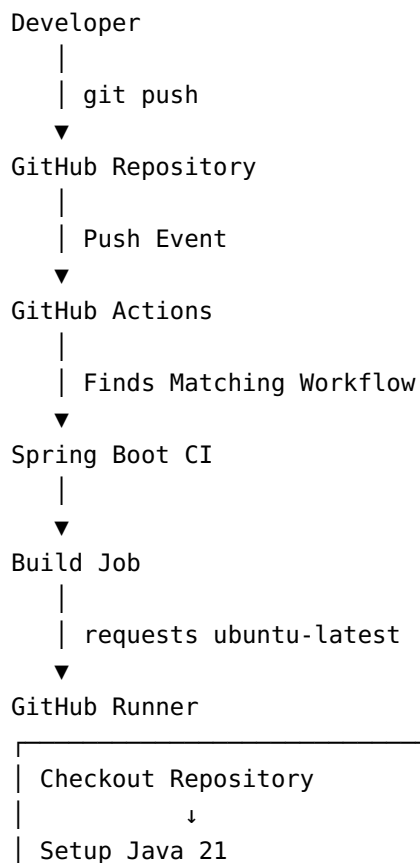
  - name: Checkout Repository
    uses: actions/checkout@v7

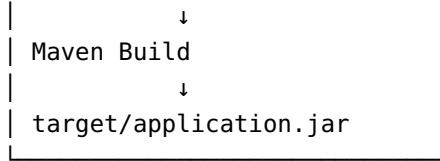
  - name: Setup Java
    uses: actions/setup-java@v6
    with:
      distribution: temurin
      java-version: '21'

  - name: Build Application
    run: ./mvnw clean package
```

19. What Happens After `git push`?

Internally, the process looks like this:





The workflow configuration itself lives inside the repository:

```
spring-demo/
├── src/
├── pom.xml
├── .github/
│   └── workflows/
│       └── ci.yml
```

This means both the application and its automation configuration are version controlled.

20. One Job vs Multiple Jobs

Consider:

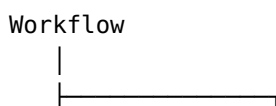
```
jobs:

  test:
    runs-on: ubuntu-latest
    steps:
      - run: ./mvnw test

  lint:
    runs-on: ubuntu-latest
    steps:
      - run: ./mvnw verify
```

Do not imagine this as one Ubuntu machine.

Think:





Each job is a runner-level execution unit.

Independent jobs can run in parallel.

Dependencies between jobs can be created using:

```
needs :
```

For example:

```
Build
  ↓
Deploy
```

21. What Happens to Files Created on a Runner?

Suppose Maven generates:

```
target/demo.jar
```

on a GitHub-hosted runner.

Once the runner/job disappears, we should not treat that machine as permanent storage.

A production CI/CD architecture normally stores build artifacts separately.

```
Build
  ↓
Artifact
  ↓
Artifact Repository
```

↓
Deployment

Examples of artifact repositories include Nexus and Artifactory.

GitHub Actions Practical Demo

Now let's build a simple CI/CD pipeline for a Spring Boot application.

Our architecture will be:

```
Feature Branch
  ↓
PR → main
  ↓
GitHub-Hosted Runner
  ↓
Unit Tests
  ↓
PASS
  ↓
Merge PR
  ↓
Self-Hosted Runner
inside Ubuntu VM
  ↓
Build JAR
  ↓
Replace Old JAR
  ↓
Restart Spring Boot App
  ↓
Deployment Complete
```

This demonstrates both:

```
CI → Test before merging
CD → Deploy after merging
```

22. Create the Spring Boot Application

Create a basic Spring Boot application with:

- A controller
- A `/hello` endpoint
- Unit/integration tests
- Maven Wrapper

Example project:

```
github-actions-demo/  
|  
├─ src/  
├─ pom.xml  
├─ mvnw  
└─ ...
```

23. Push the Project to GitHub

Create a GitHub repository such as:

```
github-actions-demo
```

Then initialize and push the project:

```
git init  
  
git add .  
  
git commit -m "initial spring boot application"  
  
git branch -M main  
  
git remote add origin <YOUR-GITHUB-REPOSITORY>  
  
git push -u origin main
```

24. PR Testing Workflow

Create:

```
.github/workflows/pr-tests.yml
```

Add:

```
name: PR Tests

on:
  pull_request:
    branches:
      - main

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
        uses: actions/checkout@v7

      - name: Setup Java
        uses: actions/setup-java@v6
        with:
          distribution: temurin
          java-version: '21'

      - name: Run Unit Tests
        run: |
          chmod +x mvnw
          ./mvnw test
```

The flow is:

```
PR Targeting main
  ↓
pull_request Event
  ↓
```

```
Test Job Created
  ↓
GitHub-Hosted Ubuntu Runner
  ↓
Checkout Repository
  ↓
Configure Java 21
  ↓
./mvnw test
```

Notice:

```
runs-on: ubuntu-latest
```

This runner belongs to GitHub.

It is **not** our Ubuntu Multipass VM.

25. CI and Deployment Workflows

For a clean demo, keep both workflow files on `main` before starting the actual PR demonstration.

Initial `main` can contain:

```
Application
Tests
PR Workflow
Deployment Workflow
```

During the demonstration, only modify application code and tests.

This keeps the focus on CI/CD rather than workflow setup.

26. Prepare the Ubuntu Machine

Our Ubuntu VM will act as the deployment server and self-hosted runner.

Install Java:

```
sudo apt update  
  
sudo apt install openjdk-21-jdk -y
```

Verify:

```
java -version
```

Java is required here even though the CI workflow already configured Java.

Why?

Because they are two different machines.

```
CI  
↓  
GitHub-Hosted Runner  
↓  
Temporary Machine
```

while:

```
Production Demo  
↓  
Ubuntu VM  
↓  
Application Runs Here
```

Therefore the deployment machine needs its own Java installation.

27. Create the Application Directory

Inside Ubuntu:

```
mkdir -p /home/ubuntu/spring-app
```

Eventually the deployed application will look like:

```
/home/ubuntu/spring-app/  
├─ app.jar  
├─ app.pid  
└─ app.log
```

28. Synchronize the Ubuntu Clock

Correct system time is important when interacting with services that rely on secure authentication.

Enable NTP synchronization:

```
sudo timedatectl set-ntp true
```

Restart the time synchronization service:

```
sudo systemctl restart systemd-timesyncd
```

Check:

```
timedatectl
```

Look for:

```
System clock synchronized: yes  
NTP service: active
```

You can also check UTC time:

```
date -u
```

If `systemd-timesyncd` is unavailable:

```
sudo apt update  
sudo apt install systemd-timesyncd -y
```

Then enable synchronization again.

29. Register Ubuntu as a Self-Hosted Runner

Open the repository:

```
Repository
↓
Settings
↓
Actions
↓
Runners
↓
New self-hosted runner
```

Select:

```
Linux
```

Check the architecture of the Ubuntu VM:

```
uname -m
```

If the output is:

```
aarch64
```

use:

```
ARM64
```

This is common when Multipass is running on Apple Silicon.

If the output is:

```
x86_64
```

use:

```
x64
```

GitHub will generate the current installation commands for the runner.

Run those commands inside:

```
/home/ubuntu
```

Do not copy old runner-version commands from tutorials.

30. Configure the Runner

During configuration, give the runner a name such as:

```
multipass-runner
```

The default work directory can remain:

```
_work
```

After successful registration, GitHub should report that the runner has been added and connected.

Start the runner using the command provided by GitHub, typically:

```
./run.sh
```

The terminal should display something similar to:

```
Listening for Jobs...
```

Keep this terminal running.

The runner is now waiting for GitHub to send it jobs.

31. Verify the Runner

Open:

```
Repository  
→ Settings  
→ Actions  
→ Runners
```

You should see something similar to:

```
multipass-runner  
  
Idle
```

Idle means:

The runner is connected and waiting for a job.

32. Runner Terminal vs Normal Ubuntu Terminal

Keep the first terminal dedicated to the runner:

```
Terminal 1  
  
/home/ubuntu/actions-runner  
./run.sh  
  
Listening for Jobs...
```

Open another terminal and enter the VM:

```
multipass shell devops
```

This second terminal can be used for inspecting files, logs and the running application.

33. Deployment Workflow

Back inside the Spring Boot project on the Mac, create:

```
.github/workflows/deploy.yml
```

Example:

```
name: Deploy To Ubuntu

on:
  pull_request:
    branches:
      - main
    types:
      - closed

jobs:
  deploy:
    if: github.event.pull_request.merged == true
    runs-on: self-hosted

    steps:
      - name: Checkout Main
        uses: actions/checkout@v7
        with:
          ref: main

      - name: Build Application
        run: |
          chmod +x mvnw
          ./mvnw clean package -DskipTests

      - name: Stop Old Application
        run: |
          if [ -f /home/ubuntu/spring-app/app.pid ]; then
            PID=$(cat /home/ubuntu/spring-app/app.pid)

            if kill -0 "$PID" 2>/dev/null; then
              kill "$PID"
              sleep 2
```

```

        fi

        rm -f /home/ubuntu/spring-app/app.pid
    fi

    - name: Copy New Application
      run: |
        cp target/app.jar /home/ubuntu/spring-app/app.jar

    - name: Start Application
      run: |
        cd /home/ubuntu/spring-app

        RUNNER_TRACKING_ID="" nohup java -jar app.jar
        > app.log 2>&1 &

        echo $! > app.pid

    - name: Verify Deployment
      run: |
        sleep 5
        curl --fail http://localhost:8080/hello

```

The JAR filename must match the artifact generated by your Maven configuration.

34. When Does Deployment Run?

The workflow uses:

```

on:
  pull_request:
    branches:
      - main
    types:
      - closed

```

This means:

React whenever a pull request targeting `main` is closed.

However, a PR can also be closed without merging.

Therefore:

```
if: github.event.pull_request.merged == true
```

ensures deployment happens only after a successful merge.

```
PR Closed  
but not merged  
↓  
No Deployment
```

```
PR Merged  
↓  
Deployment
```

35. Why Does Deployment Run Inside Multipass?

The deployment job contains:

```
runs-on: self-hosted
```

Our Ubuntu VM was previously registered as:

```
multipass-runner
```

Therefore:

```
GitHub  
↓  
Self-Hosted Runner  
↓  
multipass-runner  
↓  
Ubuntu VM
```

Every subsequent `run:` command executes directly inside that Ubuntu machine.

36. Repository Checkout on a Self-Hosted Runner

The workflow contains:

```
- name: Checkout Main
  uses: actions/checkout@v7
  with:
    ref: main
```

GitHub automatically downloads the repository into the runner's workspace.

You do **not** need to manually clone the Spring Boot project into:

```
/home/ubuntu/
```

The runner maintains its own workspace:

```
/home/ubuntu/actions-runner/_work/
```

For example:

```
/home/ubuntu/actions-runner/_work/
├─ github-actions-demo/
│   └─ github-actions-demo/
│       ├── src/
│       ├── pom.xml
│       ├── mvnw
│       └─ ...
```

This directory should normally be managed by the runner itself.

37. Workspace vs Deployment Directory

There are two different locations involved.

Runner Workspace

```
/home/ubuntu/actions-runner/_work/
```

Used temporarily by GitHub Actions to:

- Checkout source code
- Build the project
- Run workflow commands

Deployment Directory

```
/home/ubuntu/spring-app/
```

Used to keep the actual running application.

The flow becomes:

```
Runner Workspace
  ↓
Build
  ↓
target/app.jar
  ↓
COPY
  ↓
/home/ubuntu/spring-app/app.jar
```

This distinction is important.

We do not manually place source code inside the deployment directory.

38. Create a Feature Branch

On the development machine:

```
git checkout -b feature/version-2
```

Make some change to the application.

Then:

```
git add .  
  
git commit -m "update hello endpoint"  
  
git push -u origin feature/version-2
```

Create a pull request:

```
feature/version-2  
↓  
main
```

39. CI Starts Automatically

Creating the PR triggers:

```
on:  
  pull_request:  
    branches:  
      - main
```

GitHub starts a **GitHub-hosted runner**.

```
PR Tests  
↓  
Checkout  
↓  
Setup Java  
↓  
Run Unit Tests
```

If the test is intentionally broken:

```
PR Created  
↓  
CI Runs
```

↓
Test Fails ❌

At this point nothing has been deployed.

40. CI vs CD

This demo clearly separates the two concepts.

Continuous Integration

Pull Request
↓
Build / Test
↓
Validate Code Before Merge

Continuous Deployment

PR Merged
↓
Deployment Workflow
↓
Build Application
↓
Deploy to Server

So:

CI
=
Validate before merging

CD
=
Deploy after merging

41. Fix the Test and Merge the PR

Fix the failing test and push the correction.

Once the CI workflow becomes successful, merge the PR.

GitHub now receives:

```
Pull Request Closed
      +
merged = true
```

Therefore:

```
Deploy To Ubuntu
```

starts automatically.

42. Watch the Self-Hosted Runner

The Ubuntu runner terminal should initially show:

```
Listening for Jobs...
```

After merging:

```
GitHub
  ↓
multipass-runner
  ↓
Running job: deploy
```

GitHub sends the deployment job directly to the Ubuntu VM.

43. Deployment Steps

The workflow now performs:

```
Checkout Main
  ↓
Build Application
  ↓
Stop Old Application
  ↓
Copy New Application
  ↓
Start Application
  ↓
Verify Deployment
```

44. Build Application

The project is built inside the runner workspace:

```
./mvnw clean package -DskipTests
```

The resulting JAR appears under:

```
target/
```

Example:

```
/home/ubuntu/actions-runner/_work/.../target/app.jar
```

45. Stop the Old Application

The deployment stores the process ID in:

```
/home/ubuntu/spring-app/app.pid
```

Before starting a new version, the workflow checks whether an old process exists.

If it exists:

```
Read PID
↓
Stop Process
↓
Remove app.pid
```

This prevents multiple application instances from accidentally running on the same port.

46. Copy the New JAR

The newly generated application is copied from the runner workspace:

```
_work/.../target/app.jar
```

to:

```
/home/ubuntu/spring-app/app.jar
```

So the source-code/build workspace and application runtime directory remain separate.

47. Start the Application

The workflow changes to the deployment directory:

```
cd /home/ubuntu/spring-app
```

and starts Spring Boot:

```
RUNNER_TRACKING_ID="" nohup java -jar app.jar
> app.log 2>&1 &
```

The process ID is saved:

```
echo $! > app.pid
```

Now:

```
app.jar
```

continues running in the background.

Logs are written to:

```
app.log
```

and the PID is stored in:

```
app.pid
```

48. Verify Deployment

The final workflow step checks the application:

```
sleep 5  
curl --fail http://localhost:8080/hello
```

If the endpoint responds successfully, the deployment job passes.

The GitHub Actions page should show:

- ✓ Checkout Main
- ✓ Build Application
- ✓ Stop Old Application
- ✓ Copy New Application
- ✓ Start Application
- ✓ Verify Deployment

49. Verify the Application Manually

You can also test the application using the Ubuntu VM's IP address.

Example:

```
http://<VM-IP>:8080/hello
```

If the new response appears, the latest version has successfully been deployed.

Complete Architecture

The complete flow is:

```
Developer
  ↓
Feature Branch
  ↓
Pull Request
  ↓
GitHub Actions
  ↓
GitHub-Hosted Runner
  ↓
Run Tests
  ↓
Tests Pass
  ↓
Merge to main
  ↓
Deployment Workflow
  ↓
Self-Hosted Runner
  ↓
Ubuntu VM
  ↓
Checkout main
  ↓
Build JAR
  ↓
Stop Old Application
  ↓
Copy New JAR
  ↓
Start Application
  ↓
Verify Endpoint
```

GitHub-Hosted vs Self-Hosted in This Demo

Purpose	Runner
PR Testing	GitHub-hosted
Spring Boot Unit Tests	GitHub-hosted
Deployment	Self-hosted
Actual Spring Boot Runtime	Ubuntu VM

This is an important architectural distinction.

The CI machine does not have to be the deployment machine.

Final Mental Model

GitHub Actions can be understood using one simple hierarchy:

```
EVENT
↓
WORKFLOW
↓
JOB
↓
RUNNER
↓
STEPS
↓
COMMANDS / ACTIONS
```

For our practical example:

```
PR Created
↓
PR Tests Workflow
↓
Test Job
↓
GitHub-Hosted Runner
```

↓
Unit Tests

and:

PR Merged
↓
Deploy Workflow
↓
Deploy Job
↓
Self-Hosted Runner
↓
Ubuntu VM
↓
Spring Boot Deployment

That is the core idea behind using GitHub Actions for a complete CI/CD pipeline.